

MIDTERM1 PROJECT

TEST BENCH



ZEINAB AZORDEH

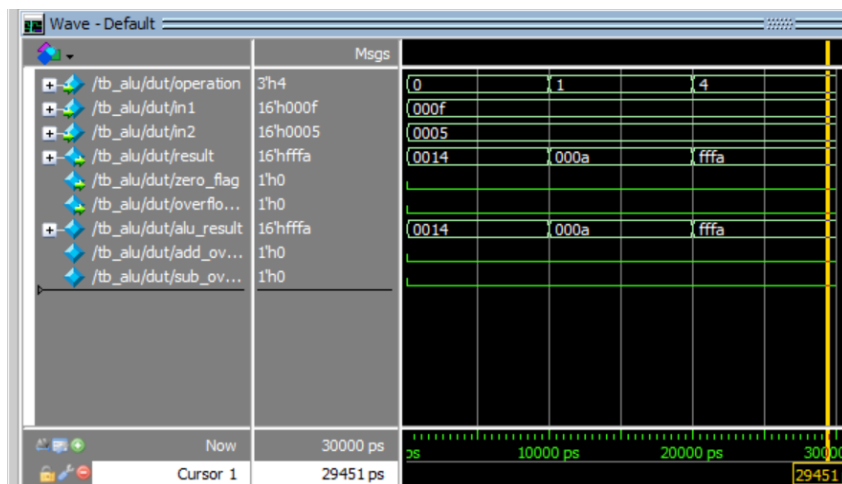
KNTU
40317653

TESTING EACH MODULE SEPERATLY:

ALU:

Here I only tested, add and sub and not, because I did not know I had to test each module separately and I am adding this last minute, also I already know my alu works fine, so it is rather a short test:

We give inputs a, b, and the operation, then take outputs: result, zero_flag, and overflow_flag.



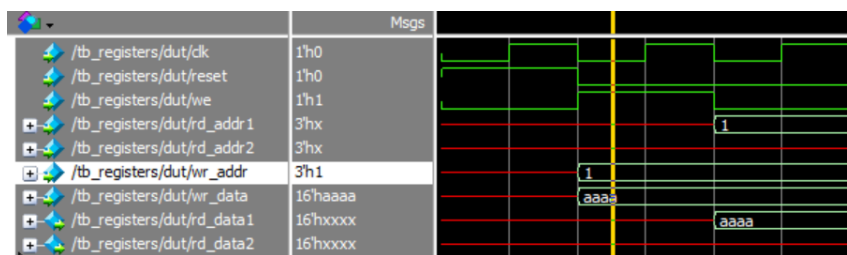
1. $15 + 5 = 20$ (0014 hex)

2. $15 - 5 = 10$ (000a hex)

3. $\text{Not}(0005) = (\text{fffa})\text{hex}$

REGISTER:

We are going to check the read enable and write enable, in our registers. Metaphorically the register.v file is like a desk, which we use to hold things temporarily.



```
initial begin
    clk = 0; reset = 1; we = 0; #10;

    reset = 0; we = 1;      // Turn off reset, turn on write
    w_addr = 3'd1;          // Choose Register 1
    w_data = 16'hAAAA; #10; // Write the word "AAAA"

    we = 0; r_addr1 = 3'd1; #10; // Read from Register 1
```

We just write AAAA to reg1 then read from reg1 and as you can see in the wave form, the result is AAAA.

MEMORY:

Memory is like registers; we just check read and write:

```
initial begin
    clk = 0; reset = 1; we = 0; #10;

    reset = 0; we = 1;
    addr = 12'h050;          // Choose address 0x050
    data_in = 16'h1234; #10; // Write "1234"

    we = 0; #10;             // Turn off write, just read
```

Simply we choose address 80(50 hex) to write inside then the output will be 1234 since the address is still chosen.

	Msgs	
/tb_memory/dut/clk	1'h1	
/tb_memory/dut/reset	1'h0	
/tb_memory/dut/we	1'h0	
/tb_memory/dut/addr	12'h050	050
/tb_memory/dut/data_in	16'h1234	1234
/tb_memory/dut/data_out	16'h1234	0000 1234
/tb_memory/dut/i	32'h00001000	00001000

CONTROL:

Control's file has way too many things to do. Since I knew this last minute to add, I unfortunately do not have time to test all the things the controller can do. So I chose add immediate and load to test:

```
z_flag = 0; pc = 0;

// Test ADDI (Opcode: 1100)
inst = 16'b1100_0000_0000_0101; #10;

// Test LOAD (Opcode: 0000)
inst = 16'b0000_0000_0000_0000; #10;
```

Control module as I described in the last pdf, is a boss that sets the right signals to work through, so to check if it works right I have to explain all signals tied to one instruction.

1. The ADDI 5 Command (Left Side)

The instruction is c005 (Add Immediate). The Control Unit sets the signals to calculate a new number:

- **alu_src_sel = 1:** Tells the ALU to use the number 5 directly from the instruction.
- **alu_op = 0:** Tells ALU to perform addition.
- **reg_write_en = 1:** Enables the Register File to save the new answer.
- **reg_src_sel = 0:** Routes the ALU's mathematical result directly into the registers.
- **Memory Signals = 0:** Keeps the memory turned off because it isn't needed for this math problem.

2. The LOAD 0 Command (Right Side)

The instruction changes to 0000 (Load from Memory). The Control Unit sets the signals to move data from the Memory into a Register:

- **mem_read_en = 1:** Instructs the Memory module to output its stored data.
- **reg_write_en = 1:** Enables the Register File to save the incoming data.
- **reg_src_sel = 1:** This is the most important switch. It routes the data from the Memory directly into the registers, telling the system to ignore the ALU.
- **alu_op = 5:** Puts the ALU into an idle "pass-through" state since it is not doing any calculations.

	Msgs		
/tb_control/dut/instruction	16hc005	c005	0000
/tb_control/dut/zero_flag	1'h0		
/tb_control/dut/pc_current	16'h0000	0000	
/tb_control/dut/reg_write_en	1'h1		
/tb_control/dut/reg_write_addr	3'h0	0	
/tb_control/dut/reg_src_sel	2'h0	0	1
/tb_control/dut/alu_src_sel	1'h1		
/tb_control/dut/alu_op	3'h0	0	5
/tb_control/dut/mem_read_en	1'h0		
/tb_control/dut/mem_write_en	1'h0		
/tb_control/dut/pc_next_sel	2'h0	0	
/tb_control/dut/branch_addr	12'h005	005	000
/tb_control/dut/reg_read_addr1	3'h0	0	
/tb_control/dut/reg_read_addr2	3'h0	0	
/tb_control/dut/opcode	4'hc	c	0
/tb_control/dut/reg_index	3'h0	0	
/tb_control/dut/addr_12	12'h005	005	000
/tb_control/dut/addr_9	9'h005	005	000
/tb_control/dut/func	9'h005	005	000

Testbench Explanation

Tasks are like functions. We write them once and use them throughout our code. I have implemented:

1. `load_instruction`: puts a 16-bit command directly into the processor's memory so it is ready to be read.
2. `init_register`: puts a starting number directly into a register (like putting a paper on a worker's desk) before a test begins.
3. `check_register/memory`: These two tasks are the judges. After the processor does math, these tasks look at the final answer. If the answer is correct, they print "[PASS]" and add 1 point to the `pass_count`. If it is wrong, they print "[FAIL]" and add 1 point to the `fail_count`.
4. `run_cycles`: tells the testbench to wait while the processor works. The #1 at the end waits for one tiny extra nanosecond. This stops the testbench from checking the answer too early before the processor finishes saving it.

The Main Script:

```
// ===== MAIN TEST SEQUENCE =====
initial begin
    test_count = 0;
    pass_count = 0;
    fail_count = 0;
```

This is how every test is written.

1. We load the instructions (like `ADDI 5`) into memory.
2. We use `run_cycles(1)` to let the clock tick one time, so the processor does the work.
3. We use `check_register` to see if Register 0 finally has the number 5 (`16'h0005`) inside it.

Test1:

```
// Test 1a: ADDI 5 (R0 = 0 + 5)
load_instruction(0, 16'b1100_000000000101);
run_cycles(1);
check_register(1, 0, 16'h0005);

// Test 1b: ADDI 10 (R0 = 5 + 10)
load_instruction(1, 16'b1100_000000001010);
run_cycles(1);
check_register(2, 0, 16'h000F);

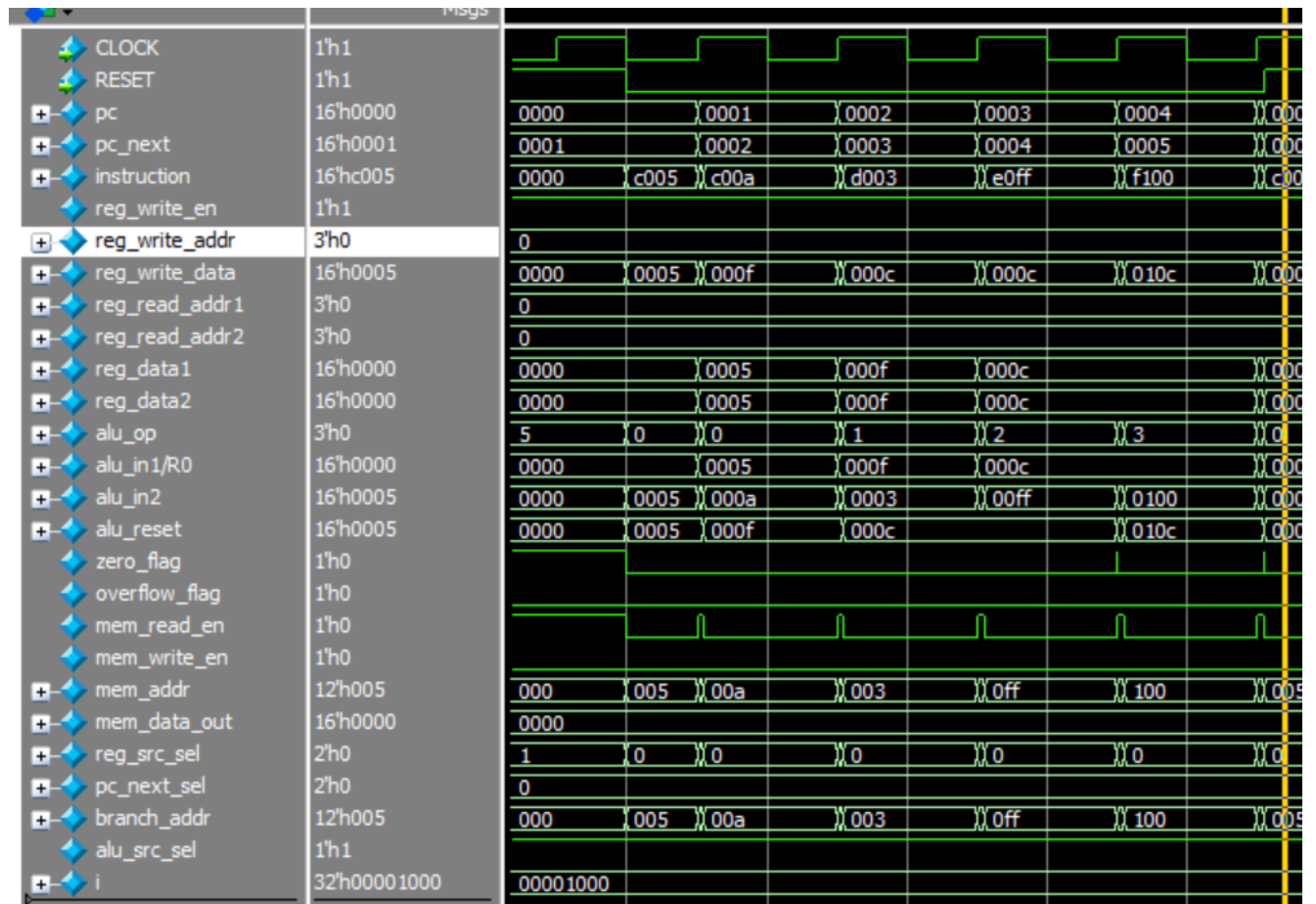
// Test 1c: SUBI 3 (R0 = 15 - 3)
load_instruction(2, 16'b1101_000000000011);
run_cycles(1);
check_register(3, 0, 16'h000C);

// Test 1d: ANDI 0xFF (Mask to lower 8 bits)
load_instruction(3, 16'b1110_000011111111);
run_cycles(1);
check_register(4, 0, 16'h000C);

// Test 1e: ORI 0x100 (Set bit 8)
load_instruction(4, 16'b1111_000100000000);
run_cycles(1);
check_register(5, 0, 16'h010C);
```

The output is visible either from the testbench or from the terminal:

```
#
# --- TEST 1: Immediate Operations (ADDI, SUBI, ANDI, ORI) ---
#
# [PASS] Test      1: R0 = 0x0005
# [PASS] Test      2: R0 = 0x000f
# [PASS] Test      3: R0 = 0x000c
# [PASS] Test      4: R0 = 0x000c
# [PASS] Test      5: R0 = 0x010c
#
```



Test2:

```
// ===== TEST 2: REGISTER OPERATIONS =====
$display("\n--- TEST 2: Register Operations (MOVEIO, MOV

reset = 1;
#20;
reset = 0;

init_register(0, 16'h0000);
init_register(1, 16'h0008);
init_register(2, 16'h0003);

// Test 2a: MOVEFROM R1 (R0 = R1 = 8)
load_instruction(0, 16'b1000_001_000000010);
run_cycles(1);
check_register(6, 0, 16'h0008);

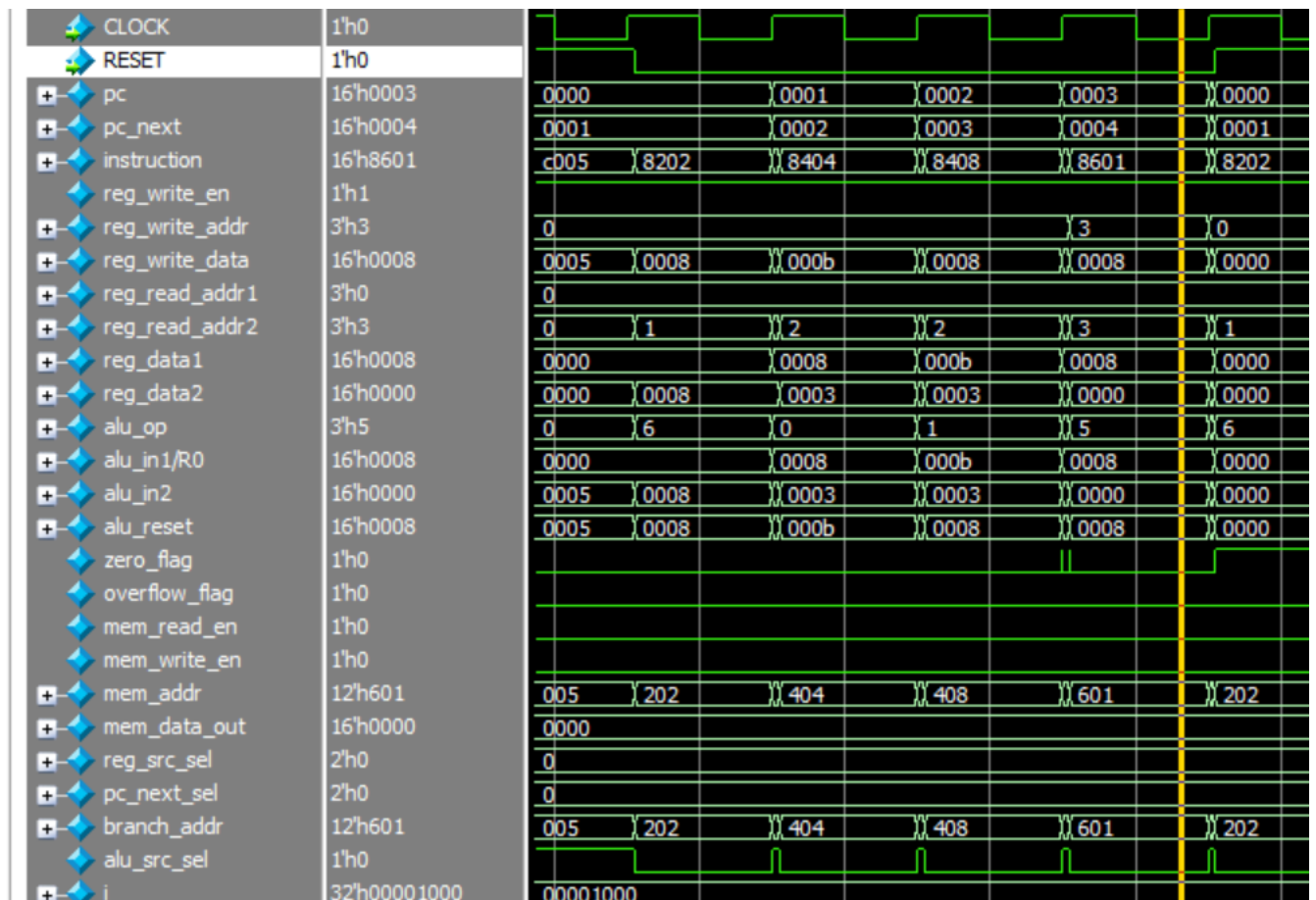
// Test 2b: ADD R2 (R0 = 8 + 3 = 11)
load_instruction(1, 16'b1000_010_000000100);
run_cycles(1);
check_register(7, 0, 16'h000B);

// Test 2c: SUB R2 (R0 = 11 - 3 = 8)
load_instruction(2, 16'b1000_010_000000100);
run_cycles(1);
check_register(8, 0, 16'h0008);

// Test 2d: MOVEIO R3 (R3 = R0 = 8)
load_instruction(3, 16'b1000_011_000000001);
run_cycles(1);
check_register(9, 3, 16'h0008);
```

All the signals and data are visible to see

```
--- TEST 2: Register Operations (MOVEIO, MOVEFROM, ADD, SUB) ---
[PASS] Test      6: R0 = 0x0008
[PASS] Test      7: R0 = 0x000b
[PASS] Test      8: R0 = 0x0008
[PASS] Test      9: R3 = 0x0008
```



Test3:

```
// ===== TEST 3: BITWISE OPERATIONS =====
$display("\n--- TEST 3: Bitwise Operations (AND, OR, NOT) ---");

reset = 1;
#20;
reset = 0;

init_register(0, 16'h00FF);
init_register(1, 16'h0F0F);
init_register(2, 16'h00AA);

// Test 3a: AND R1 (R0 = 0x00FF AND 0x0F0F = 0x000F)
load_instruction(0, 16'b1000_001_000010000);
run_cycles(1);
check_register(10, 0, 16'h000F);

// Test 3b: OR R2 (R0 = 0x000F OR 0x00AA = 0x00AF)
load_instruction(1, 16'b1000_010_000100000);
run_cycles(1);
check_register(11, 0, 16'h00AF);

// Test 3c: NOT R1 (R0 = NOT 0x0F0F = 0xF0F0)
load_instruction(2, 16'b1000_001_001000000);
run_cycles(1);
check_register(12, 0, 16'hF0F0);
```

--- TEST 3: Bitwise Operations (AND, OR, NOT) ---

[PASS] Test 10: R0 = 0x000f
 [PASS] Test 11: R0 = 0x00af
 [PASS] Test 12: R0 = 0xf0f0

	Msgs	
CLOCK	1'h0	
RESET	1'h0	
pc	16'h0003	0000
pc_next	16'h0004	0001
instruction	16'h8601	8202
reg_write_en	1'h1	
reg_write_addr	3'h3	0
reg_write_data	16'h0008	0000
reg_read_addr1	3'h0	0
reg_read_addr2	3'h3	1
reg_data1	16'h0008	0000
reg_data2	16'h0000	0000
alu_op	3'h5	6
alu_in1/R0	16'h0008	0000
alu_in2	16'h0000	0000
alu_reset	16'h0008	0000
zero_flag	1'h0	
overflow_flag	1'h0	
mem_read_en	1'h0	
mem_write_en	1'h0	
mem_addr	12'h601	202
mem_data_out	16'h0000	0000
reg_src_sel	2'h0	0
pc_next_sel	2'h0	0
branch_addr	12'h601	202
alu_src_sel	1'h0	
i	32'h00001000	00001000

Test4:

```
// ===== TEST 4: MEMORY OPERATIONS =====
$display("\n--- TEST 4: Memory Operations (LOAD, STORE) ---\n");

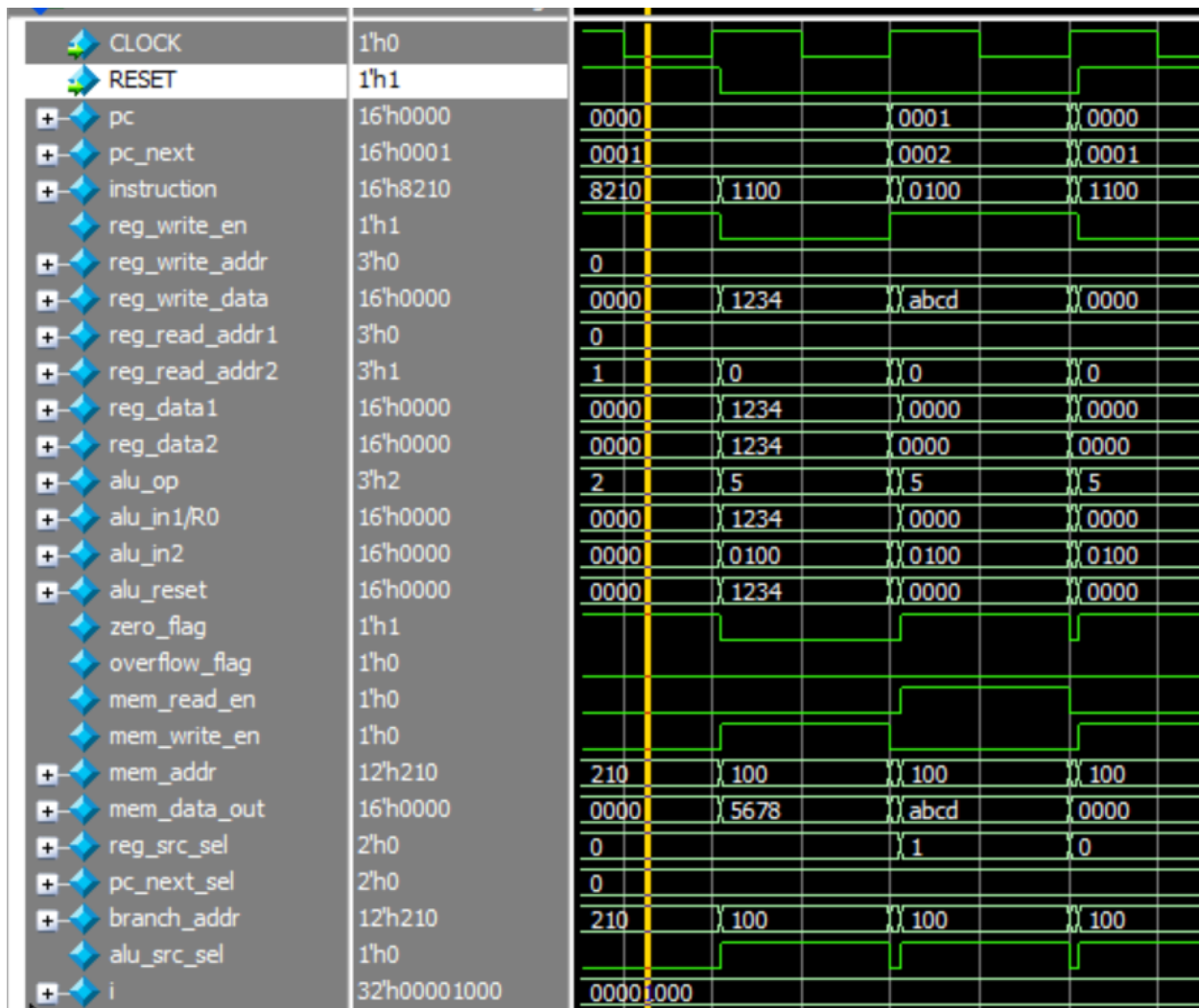
reset = 1;
#20;
reset = 0;

init_register(0, 16'h1234);
dut.data_mem.mem[12'h100] = 16'h5678;

// Test 4a: STORE 0x100 (M[0x100] = R0)
load_instruction(0, 16'b0001_000100000000);
run_cycles(1);
check_memory(13, 12'h100, 16'h1234);

// Test 4b: LOAD 0x100 (R0 = M[0x100])
init_register(0, 16'h0000);
dut.data_mem.mem[12'h100] = 16'hABCD;
load_instruction(1, 16'b0000_000100000000);
run_cycles(1);
check_register(14, 0, 16'hABCD);
```

```
--- TEST 4: Memory Operations (LOAD, STORE) ---
[PASS] Test      13: M[0x100] = 0x1234
[PASS] Test      14: R0 = 0xabcd
```



Test5&6:

```
// ===== TEST 5: JUMP INSTRUCTION =====
$display("\n--- TEST 5: Control Flow (JUMP, NOP) ---\n");

reset = 1;
#20;
reset = 0;

// Test 5a: NOP (no-operation)
load_instruction(0, 16'b1000_000_010000000);
run_cycles(1);
$display("[PASS] Test 15: NOP executed (PC = 0x%04h)", dut.pc);
pass_count = pass_count + 1;
test_count = test_count + 1;

// Test 5b: JUMP to address 0x050
load_instruction(1, 16'b0010_000001010000);
run_cycles(1);
if (dut.pc == 16'h0050) begin
    $display("[PASS] Test 16: JUMP executed (PC = 0x%04h)", dut.pc);
    pass_count = pass_count + 1;
end else begin
    $display("[FAIL] Test 16: Expected PC=0x0050, got PC=0x%04h", dut.pc);
    fail_count = fail_count + 1;
end
test_count = test_count + 1;

// ===== TEST 6: BRANCHZ INSTRUCTION =====
$display("\n--- TEST 6: Conditional Branch (BRANCHZ) ---\n");

reset = 1;
#20;
reset = 0;

init_register(0, 16'h0000); // R0 = 0
init_register(1, 16'h0000); // R1 = 0

// Test 6a: BRANCHZ with equal values (should branch)
load_instruction(0, 16'b0100_001_000001000);
run_cycles(1);
if (dut.pc[8:0] == 9'h008) begin
    $display("[PASS] Test 17: BRANCHZ condition true, PC[8:0] = 0x%03h", dut.pc[8:0]);
    pass_count = pass_count + 1;
end else begin
    $display("[FAIL] Test 17: BRANCHZ expected PC[8:0]=0x008, got 0x%03h", dut.pc[8:0]);
    fail_count = fail_count + 1;
end
test_count = test_count + 1;

// Test 6b: BRANCHZ with different values (should not branch)
reset = 1;
#20;
reset = 0;

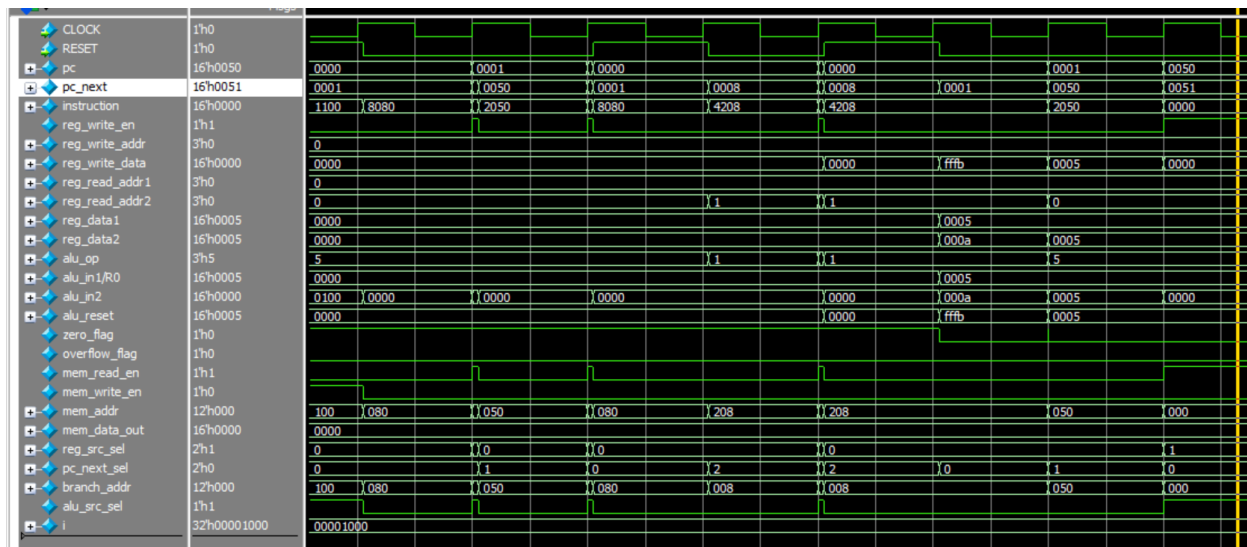
init_register(0, 16'h0005); // R0 = 5
init_register(1, 16'h000A); // R1 = 10
load_instruction(0, 16'b0100_001_000001000);
run_cycles(1);
if (dut.pc == 16'h0001) begin
    $display("[PASS] Test 18: BRANCHZ condition false, PC = 0x%04h", dut.pc);
    pass_count = pass_count + 1;
end else begin
    $display("[FAIL] Test 18: BRANCHZ expected PC=0x0001, got PC=0x%04h", dut.pc);
    fail_count = fail_count + 1;
end
test_count = test_count + 1;
```

--- TEST 5: Control Flow (JUMP, NOP) ---

[PASS] Test 15: NOP executed (PC = 0x0001)
[PASS] Test 16: JUMP executed (PC = 0x0050)

--- TEST 6: Conditional Branch (BRANCHZ) ---

[PASS] Test 17: BRANCHZ condition true, PC[8:0] = 0x008
[PASS] Test 18: BRANCHZ condition false, PC = 0x0001



THE WHOLE TEST:

```
=====
TEST SUMMARY
=====
Total Tests:      18
Passed:           18
Failed:           0

>>> ALL TESTS PASSED <<<
=====
```

Also, professor's judging system:

```
ModelSim> vsim -voptargs="+acc" work.processor_tb
# vsim -voptargs="+acc" work.processor_tb
# Start time: 10:41:14 on Jun 12, 2026
# ** Note: (vsim-8009) Loading existing optimized design_opt1
# Loading work.processor_tb(fast)
# Loading work.processor(fast)
# Loading work.control(fast)
# Loading work.registers(fast)
# Loading work.Alu(fast)
# Loading work.memory(fast)
VSIM 2> run -all
# >>>> FINAL RESULT: PASS <<<<
# R1 successfully holds 10!
# ** Note: $finish : C:/Users/zeinab/Documents/ComputerArchitecture/verilog/projects/single_cycle_processor/single_cycle_tb.v(53)
# Time: 150 ns Iteration: 0 Instance: /processor_tb
```

Thanks to a clean structure and modeling correctly.

